

15.9

Microservices Testing Strategies

Test pyramid from unit to E2E

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

The Test Pyramid

Testing microservices requires a layered approach. The Test Pyramid tells you how many tests of each type to have: MANY fast unit tests at the base, FEW slow E2E tests at the top. Each layer catches different bugs.

Level	Type	Speed	Count	Tools
5 (top)	End-to-End (E2E)	Minutes	~10-20	Playwright, Selenium
4	Contract (Pact)	Seconds	~100-200	Pact JVM
3	Component	Seconds	~200-500	Docker Compose
2	Integration	Seconds	~1,000-2,000	Testcontainers
1 (base)	Unit	Milliseconds	~5,000-10,000	JUnit 5 + Mockito

Level 1: Unit Tests

Test a single class in isolation. Mock all dependencies.

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository repo;
    @InjectMocks OrderService svc;

    @Test void placingOrderSavesIt() {
        svc.placeOrder(new PlaceOrderCommand(...));
        verify(repo).save(any(Order.class));
    }
}
```

Level 2: Integration Tests with Testcontainers

Test against REAL infrastructure (PostgreSQL, Redis, Kafka) running in Docker. No mocks for the database layer.

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {
    @Container
    static PostgreSQLContainer postgres = new
        PostgreSQLContainer<>("postgres:15");

    @Test void savedOrderCanBeRetrieved() {
        // Uses REAL PostgreSQL – not H2 or mock
        Order saved = repo.save(new Order(...));
        assertThat(repo.findById(saved.getId()).isPresent());
    }
}
```

Level 3-4: Contract Tests with Pact

Consumer defines what it expects from a provider. Provider verifies it delivers that contract. This catches breaking API changes before deployment.

```
// CONSUMER defines contract
@Pact(consumer = "OrderService", provider = "PaymentService")
public RequestResponsePact createPact(PactDslWithProvider builder) {
    return builder.given("payment service is up")
        .uponReceiving("a charge request")
        .path("/charge").method("POST")
        .willRespondWith().status(200)
        .body(LambdaDsl.newJsonBody(o ->
            o.stringType("transactionId")).build())
        .toPact();
}
```

Level 5: E2E Tests

Test the whole system end-to-end. Run against a staging environment. Keep these minimal — they are slow and fragile.

Golden Rule: If a unit test can catch it, don't write an integration test for it. If an integration test can catch it, don't write an E2E test. Each level up is 10-100x slower.

■ **Real-World Example:** Shopify: 10,000+ unit tests (30s), 2,000 integration tests with Testcontainers (8 min), 200 Pact contract tests, 15 E2E Playwright tests for checkout. This pyramid keeps their 70-service deployment under 20 minutes.